



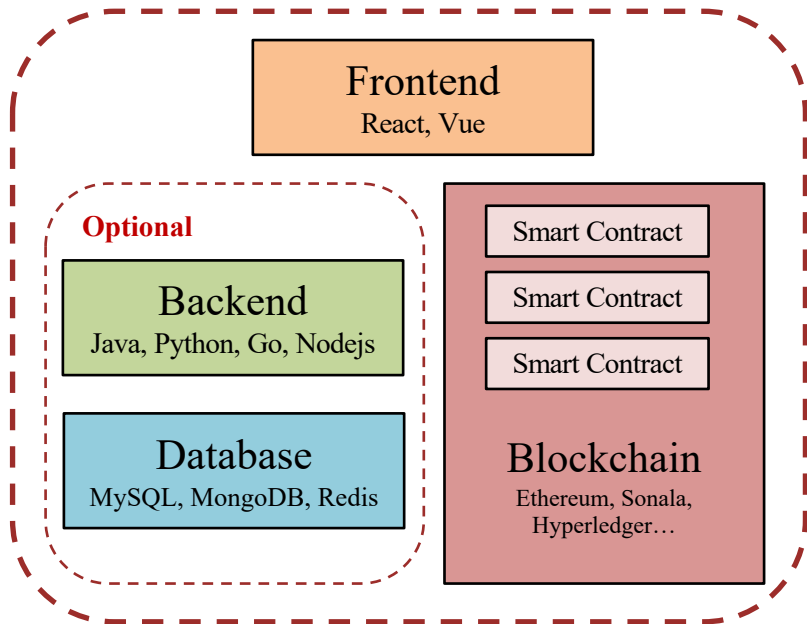
以太坊DApp开发实践

@刘轩铭 曹东204 22221263@zju.edu.cn

2022年10月



DApp怎么开发？从架构开始



- 前端界面开发（React、Vue）
- 选定一条区块链，编写对应智能合约（以太坊、Solidity）
- 智能合约部署到区块链上
- 前端调用智能合约的方法（SDK）
- （可选）用后端完成其它功能



- 一. 以太坊开发环境搭建 Node.js、npm、solc编译器，在本地调试：使用Ganache本地以太坊客户端
- 二. 以太坊编程环境搭建 Remix IDE, VS Code, Jetbrain, Truffle, hardhat
- 三. 与以太坊交互（以太坊SDK）JSON RPC 、JavaScript API、Web3.js、Metamask
- 四. DApp开发框架和流程 一个简单的Demo
- 五. DApp优秀案例 加密猫Cryptokitties, THE DAO, Maker DAO



- 一. 以太坊开发环境搭建
- 二. 以太坊编程环境搭建
- 三. 与以太坊交互（以太坊SDK）
- 四. DApp开发框架和流程
- 五. DApp优秀案例



以太坊环境搭建

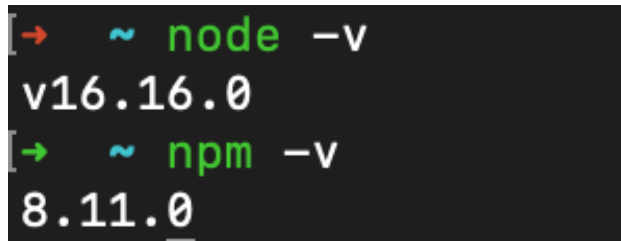
以太坊环境配置在不同操作系统的配置都比较类似，我们需要：

- Solidity编译器：依次安装node, npm, solc
 - 区块链环境：直接连接主网进行操作并不方便，所以我们安装Ganache，来搭建本地区块链环境
- 这里以MacOS为例介绍，Windows如何操作可以参考各个网站说明。**

安装nodejs、npm

npm是nodejs下的一个包管理工具，可以非常方便的安装一些基于JavaScript的软件和包。基于以太坊的很多开发工具也都是基于JavaScript来开发，可以使用npm安装。

进入<https://nodejs.org/en/download/>，网站会根据操作系统提示下载不同的nodejs版本，下载后安装即可。默认会同时安装nodejs和npm。终端执行以下命令表示安装成功：





安装solc编译器

solc是智能合约Solidity的编译器，使用以下命令安装solc:

```
npm install solc -g
```

安装成功后输入solcjs --version，出现如下界面则表示安装成功：

```
→ ~ solcjs --version  
0.8.17+commit.8df45f5f.Emscripten.clang
```



以太坊环境搭建（续）

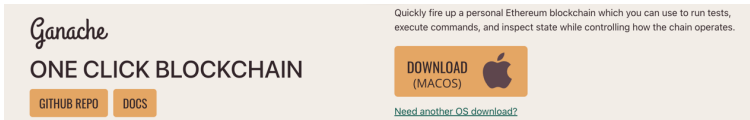
为什么要安装本地链环境？主网和测试网的开发和测试都很昂贵。

Ganache是什么：本地区块链环境，运行Ganache就相当于在本地跑了一条区块链，挖矿、交易都可以在本地独立快速完成。我们与区块链的交互可以在本地完成和测试。
但是仅仅是测试！

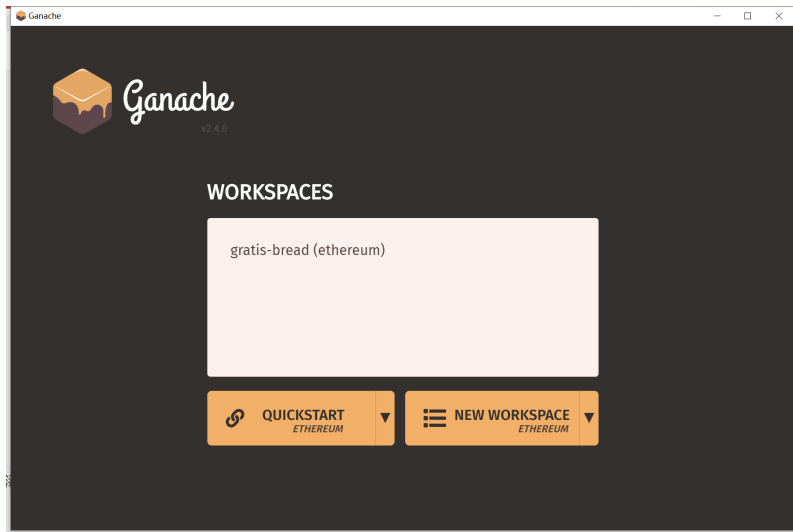
安装私有以太坊客户端Ganache（安装时间较长）

<https://trufflesuite.com/ganache/>

- 下载安装包并安装（可能时间较长）



- 运行ganache应用程序





搭建以太坊私有链

搭建一个以太坊私有链仅需要以下三个步骤：

接下来详细介绍这些步骤：

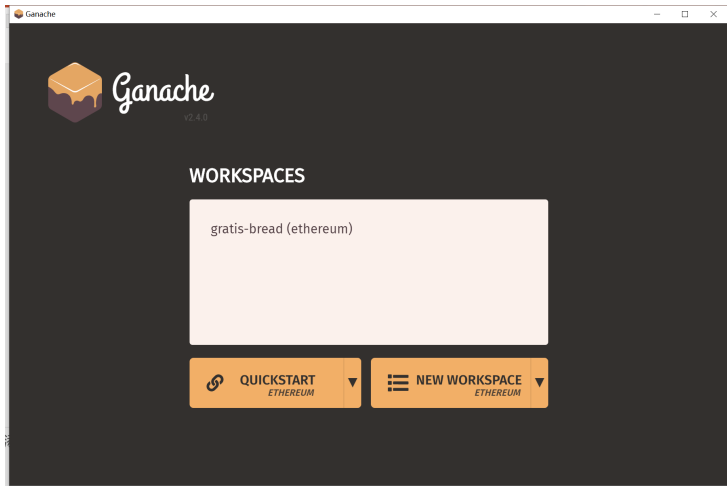
- **启动Ganache**：我们将看到右侧页面每个Workspace即为一个以太坊私有网络。可点击NEW WORKSPACE创建新的网络，也可以QUICKSTART生成默认的私有网络。



启动
Ganache

创建Workspace
或快速启动

启动私有网络

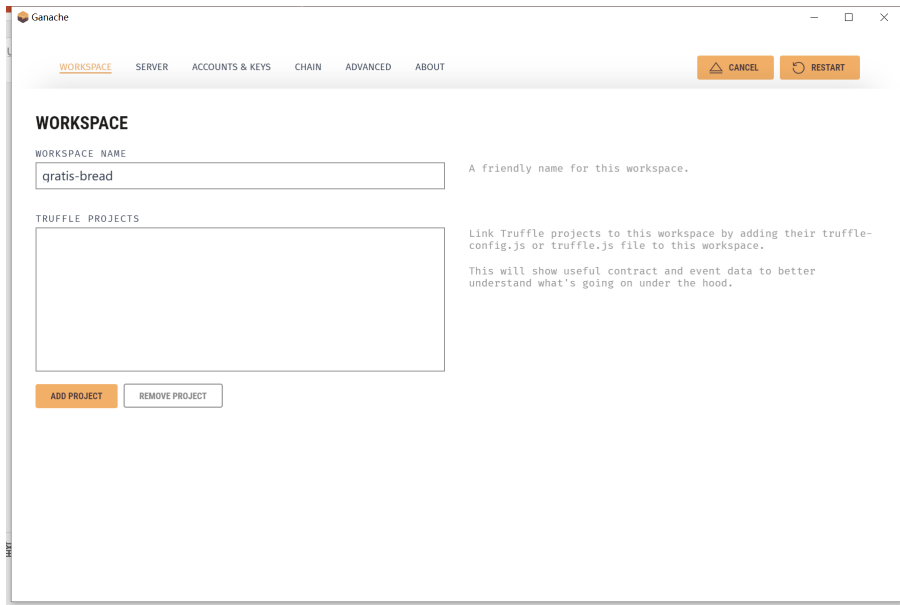




- 创建私有网络：点击已有的WORKSPACE或创建新的WORKSPACE

已创建WORKSPACE为例，将进入一个定制化页面，在这里可以配置私有网络的参数，包括API地址的端口号，链ID，默认账户数量、余额，区块的默认Gas limit和默认Gas price等。

建议RPC-URL设置为8545





搭建以太坊私有链（续）

- QUICKSTART或者保存WORKSPACE：保存并且启动网络

The screenshot shows the Ganache application interface. At the top, there are tabs for ACCOUNTS, BLOCKS, TRANSACTIONS, CONTRACTS, EVENTS, and LOGS. Below the tabs, there is a search bar and a status bar showing various metrics like CURRENT BLOCK, GAS PRICE, GAS LIMIT, HARDFORK, NETWORK ID, RPC SERVER, and MINING STATUS. The main section displays a list of accounts with their addresses, balances, transaction counts, and indices. Each account entry includes a mnemonic phrase and an HD path.

ADDRESS	BALANCE	TX COUNT	INDEX
0xA71e92e17Ed30b940A1dF38Ebe9f82D313b6d6f9	100.00 ETH	0	0
0xAEdA1f64C05eDc11a110Cc582dD62d471AC6c919	100.00 ETH	0	1
0xCE1911Ebe496164C08D2cc52F55ecc5898357fa0	100.00 ETH	0	2
0x028FcC8F36BCB1448a6E3314f88Aed219A029bd4	100.00 ETH	0	3
0x5a1D0aF84979684b3bce2643ab6eCaF3f5DA49B1	100.00 ETH	0	4
0xFB5Dcc50fd8AE8F0fd12337cc557e4b321E31a86	100.00 ETH	0	5
0x00C88df76Ef5e2B89f260Df425A013A5f0dE90f8	100.00 ETH	0	6

- 左侧包含10个账户，分别对应其地址Address，余额Balance，进行过的交易数量TX COUNT和对应的私钥（钥匙图标，用于签名发起交易）
- 上方有当前区块链的配置情况，注意RPC-SERVER是一个本地的端口URL，我们与本地区块链的交互通过它完成
- 上方还包括区块信息、交易信息、部署合约的信息以及合约发出的事件和日志
- 在开发中，我们主要关注ACCOUNTS、TRANSACTION和CONTRACTS板块



查看区块与交易信息

Ganache私有网络的挖矿是AUTOMINING形式，即当接收到新交易时才会自动挖出一个块，这样的好处是避免挖空块，节省计算资源。

CURRENT BLOCK	GAS PRICE	GAS LIMIT	HARDFORK	NETWORK ID	RPC SERVER	MINING STATUS	WORKSPACE
15	20000000000	6721975	MUIRGLACIER	5777	HTTP://127.0.0.1:8545	AUTOMINING	GRATIS-BREAD

BLOCK	MINED ON	GAS USED	TRANSACTIONS
15	2020-09-27 21:51:31	155641	1 TRANSACTION
14	2020-09-27 21:50:55	155641	1 TRANSACTION
13	2020-09-27 21:45:31	134693	1 TRANSACTION
12	2020-09-27 21:43:33	134693	1 TRANSACTION
11	2020-09-27 21:42:38	134657	1 TRANSACTION
10	2020-09-27 21:34:46	47626	1 TRANSACTION
9	2020-09-27 21:34:04	49459	1 TRANSACTION
8	2020-09-27 21:33:49	21000	1 TRANSACTION
7	2020-09-27 21:31:04	47626	1 TRANSACTION
6	2020-09-27 21:30:42	62626	1 TRANSACTION



- 一. 以太坊开发环境搭建
- 二. 以太坊编程环境搭建
- 三. 与以太坊交互（以太坊SDK）
- 四. DApp开发框架和流程
- 五. DApp优秀案例



remix

<https://remix.ethereum.org>



VsCode+插件



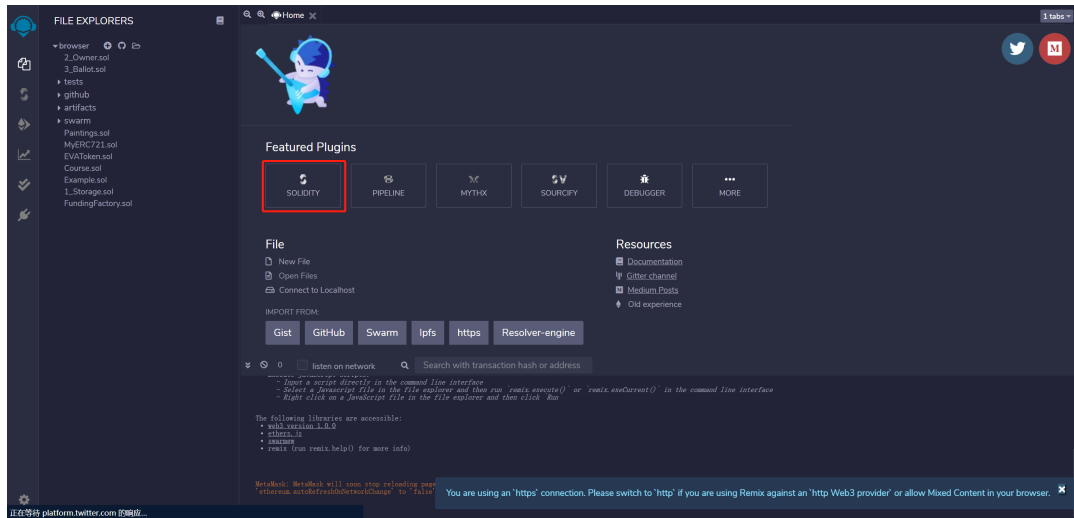
IDEA+插件



Remix是以太坊官方的DApp (去中心化应用) 线上开发环境
(WEB IDE , 有时需要科学上网)

支持功能

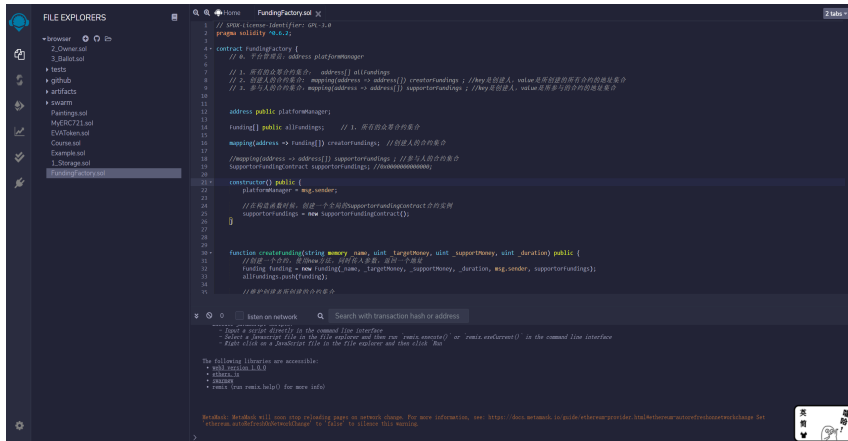
- 代码智能提示
- 一键编译&部署合约
- 可连接本地Ganache网络
- 可视化调试合约
- 代码静态分析





1、创建应用

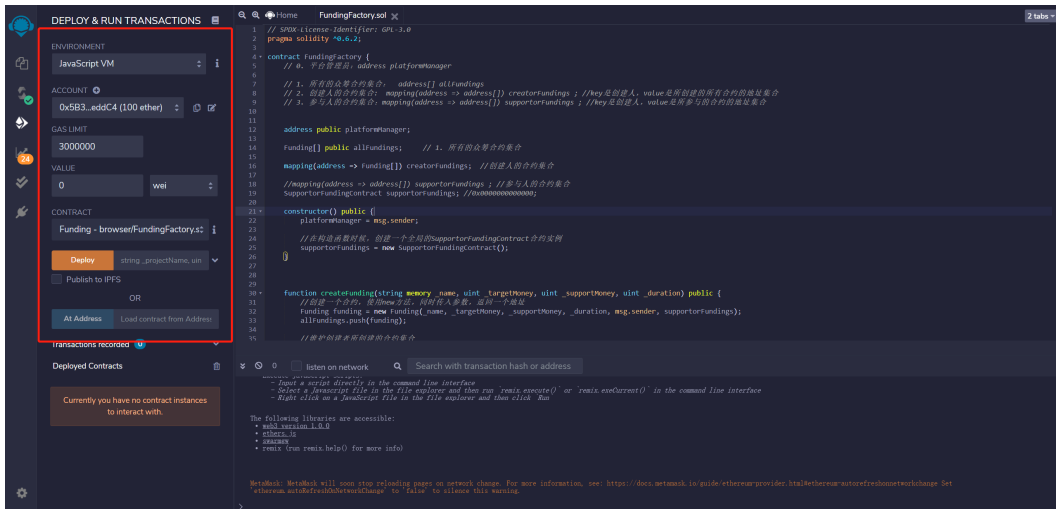
打开Remix，整体操作界面如下, 点击Solidity进入标准开发界面：





2、编译&部署合约

左侧菜单栏点击  即可进入编译页面，
选择最新的编译器等待加载（建议科学上网）





3、调试合约

左侧菜单栏点击  即可进入调试页面，这里可以选择使用本地的Ganache作为区块链网络进行合约的读写操作

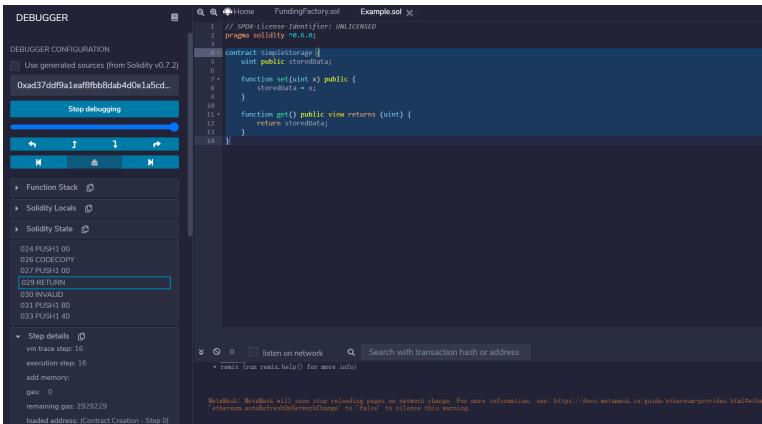
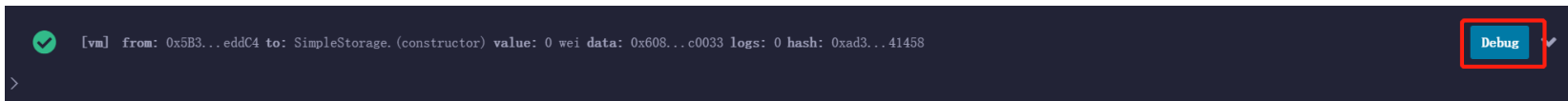
- 选择编译后的合约后点击Deploy即可部署
- 合约方法将被可视化地陈列，开发者可自由调试
- 编辑器下方的控制台可显示最新的日志信息





4. 单步Debug合约

在日志终端窗口处的每一条交易均可进行单步调试，点击Debug进入调试窗口。



在DEBUGGER窗口可以看到所有EVM执行的实时参数信息，包括：

- 字节码粒度的单步执行
- 实时堆栈、内存使用量
- 实时Gas使用量
- . . .

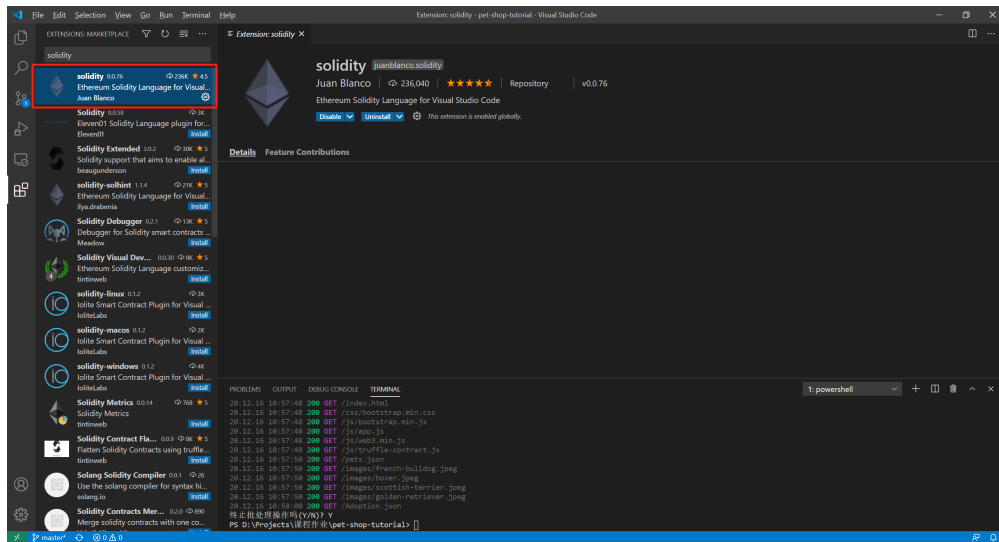


如果利用VS Code开发

安装VS Code

<https://code.visualstudio.com/>

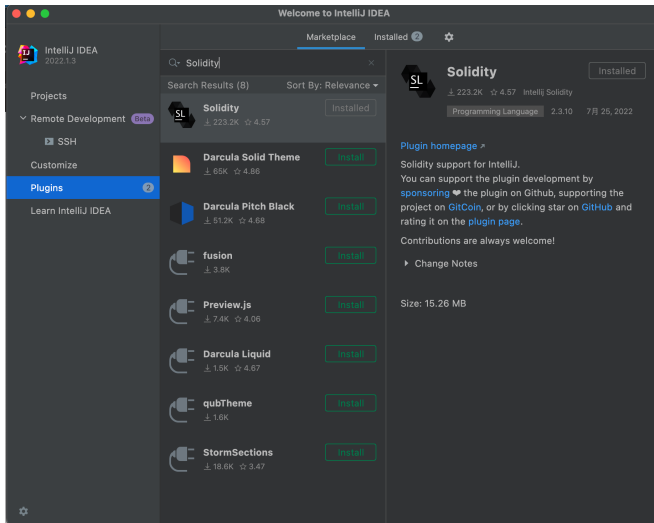
在插件市场EXTENSIONS安装solidity插件





安装JetBrain IDE和插件

以IDEA为例，在插件市场Plugins安装Solidity插件





开发框架：Truffle和Hardhat

有很多人已经帮我们封装好了比较适合开发合约、测试合约和部署合约的框架

Truffle更适合初学者、个人认为Hardhat功能更强大



二选一即可



Truffle 是一个开发工具体系, 用于以太坊平台的开发, 测试和部署. 旨在让以太坊平台开发者更轻松的编写程序。 <https://trufflesuite.com/docs/truffle/>

安装Truffle

```
npm install truffle -g
```

```
[→ ~ truffle version  
Truffle v5.5.32 (core: 5.5.32)  
Ganache v7.4.3  
Solidity v0.5.16 (solc-js)  
Node v16.16.0  
Web3.js v1.7.4
```

这里显示了Truffle的依赖工具

- Truffle 自动化工具：构建合约框架，自动编译和部署合约
- Solc-js 编译器：用来编译合约
- Node.js
- Web3：本地与区块链交互的工具



跟着文档走

<https://trufflesuite.com/docs/truffle/>



重要概念

Migration: 根据合约版本号管理部署脚本。Truffle中所有的部署合约操作均视为migrate

Unit test: Truffle 提供了使用Javascript和Solidity进行单元测试的方案，这里推荐使用Javascript编写测试用例，更具备可扩展性



Hardhat是一个方便在以太坊上进行构建的任务运行器。使用它可以帮助开发人员管理和自动化构建智能合约和DApp的过程用来部署合约，运行测试和**调试代码**。

<https://hardhat.org/docs>

安装Hardhat

```
# 新建并进入一个文件夹
mkdir my-contracts
cd my-contracts
# 初始化文件夹内的npm环境，新建package.json文件
npm init --yes
# 安装hardhat依赖
npm install -D hardhat
# 使用hardhat脚本创建工程文件
npx hardhat
```

```
[→ contracts npx hardhat
888 888 888 888
888 888 888 888
888 888 888 888
8888888888 8888b. 888d888 .d88888 88888b. 8888b. 888888
888 888 "88b 888P" d88" 888 888 "88b "88b 888
888 888 .d888888 888 888 888 888 .d888888 888
888 888 888 888 888 Y88b 888 888 888 888 Y88b.
888 888 "Y888888 888 "Y88888 888 888 "Y888888 "Y888

👤 Welcome to Hardhat v2.11.2 👤

✔ What do you want to do? · Create a TypeScript project
✔ Hardhat project root: · /Users/hinsliu/Desktop/区块链与数字货币-助教/demo/cont
✔ Do you want to add a .gitignore? (Y/n) · y
✔ Do you want to install this sample project's dependencies with npm (@nomicfoun
dation/hardhat-toolbox)? (Y/n) · y
```

安装完成后可以获得合约框架



文件目录

- contracts **我们的合约**
 - Contracts.sol
- node_modules library root
- scripts
 - deploy.ts **部署合约的脚本**
- test
 - Lock.ts **测试合约的脚本**
- typechain-types
 - .gitignore
 - hardhat.config.ts **Hardhat配置文件**
 - package.json
 - package-lock.json
 - README.md
 - tsconfig.json

编译合约文件

```
→ contracts npx hardhat compile
```

Generating typings for: 2 artifacts in dir: typechain-ty
Successfully generated 6 typings!
Compiled 2 Solidity files successfully

编译完成后生成

- artifacts
 - build-info
 - contracts
 - Contracts.sol
 - Lock.dbg.json
 - Lock.json**
 - hardhat
 - cache
 - contracts
 - node_modules library root
 - scripts
 - deploy.ts
 - test
 - Lock.ts

编译合约得到的ABI文件，
在于合约交互时使用

部署合约文件

```
→ contracts npx hardhat run ./scripts/deploy.ts
```

Lock with 1 ETH and unlock timestamp 1696142409 deployed to 0x5FbDB2315678afecb367f032d93F642f64180aa3

测试合约文件

```
contracts npx hardhat test
```

```
contracts npx hardhat test
```

Lock

Deployment

- ✓ Should set the right unlockTime (677ms)
- ✓ Should set the right owner
- ✓ Should receive and store the funds to lock
- ✓ Should fail if the unlockTime is not in the future

Withdrawals

Validations

- ✓ Should revert with the right error if called too soon
- ✓ Should revert with the right error if called from another account
- ✓ Shouldn't fail if the unlockTime has arrived and the owner calls it

Events

- ✓ Should emit an event on withdrawals

Transfers

- ✓ Should transfer the funds to the owner



hardhat.config.js/ts:关于部署的网络、编译版本等的设置

```
import {HardhatUserConfig} from "hardhat/config";  
import "@nomicfoundation/hardhat-toolbox";
```

```
const config: HardhatUserConfig = {
```

```
  // Solidity compiler version
```

```
  solidity: "0.8.9",
```

```
  networks: {
```

```
    hardhat: {},
```

网络的名字，部署时使用

```
    ganache: {
```

```
      // Change the url according to your ganache configuration
```

```
      url: 'http://localhost:8545',
```

本地网络的RPC-URL

```
      // Change these accounts private keys according to your ganache configuration.
```

```
      accounts: [
```

```
        '0x426a687e403df584ffa0123578be23aa22df7630be3cc836e1b27233a309f67f',
```

```
        '0xfb6ac705c6736aff3772c973534b100317822da23fadb3601b9a5f897888721',
```

```
        '0x857c2ed5d2d31456668bef8748d1370310a573a2aa386b201ff3d92b4971f78a',
```

需要使用的账户，私钥从ganache里获取

```
        '0x883c9d5be0244f6774539b1392aca0e040272584b2ceac286ccd9a01f730ea2e',
```

```
        '0xb13cc21621b9dcd9526aba5d6820140869be5f1ba7dfd53fe541e2784656cc51'
```

```
      ],
```

```
    },
```

```
  },
```

```
  paths: {
```

```
    sources: "./contracts",
```

```
    tests: "./test",
```

```
    cache: "./cache",
```

```
    artifacts: "./artifacts"
```

```
  },
```

```
};
```

```
export default config;
```

选择需要部署的网络: --network networkName

```
➔ contracts npx hardhat run ./scripts/deploy.ts --network ganache
```

```
Lock with 1 ETH and unlock timestamp 1696143268 deployed to 0x07c23111cB6c51C7e26f0003e1ec2A2503D86d12
```



使用hardhat/console.sol进行测试输出

```
pragma solidity ^0.6.0;  
  
import "hardhat/console.sol";  
  
contract Token {  
    //...  
}
```

```
function transfer(address to, uint256 amount) external {  
    console.log("Sender balance is %s tokens", balances[msg.sender]);  
    console.log("Trying to send %s tokens to %s", amount, to);  
  
    require(balances[msg.sender] >= amount, "Not enough tokens");  
  
    balances[msg.sender] -= amount;  
    balances[to] += amount;  
}
```

选择你要的方式打印输出

可以看到打印出来了...

```
$ npx hardhat test  
Compiling...  
Compiled 2 contracts successfully  
  
Token contract  
  Deployment  
    ✓ Should set the right owner  
    ✓ Should assign the total supply of tokens to the owner  
  Transactions  
Sender balance is 1000 tokens  
Trying to send 50 tokens to 0xead9c93b79ae7c1591b1fb5323bd777e86e150d4  
Sender balance is 50 tokens  
Trying to send 50 tokens to 0xe5904695748fe4a84b40b3fc79de2277660bd1d3  
    ✓ Should transfer tokens between accounts (373ms)  
    ✓ Should fail if sender doesn't have enough tokens  
Sender balance is 1000 tokens  
Trying to send 100 tokens to 0xead9c93b79ae7c1591b1fb5323bd777e86e150d4  
Sender balance is 900 tokens  
Trying to send 100 tokens to 0xe5904695748fe4a84b40b3fc79de2277660bd1d3  
    ✓ Should update balances after transfers (187ms)
```

更多关于hardhat，一篇不错的中文教程

<https://learnblockchain.cn/article/1356>

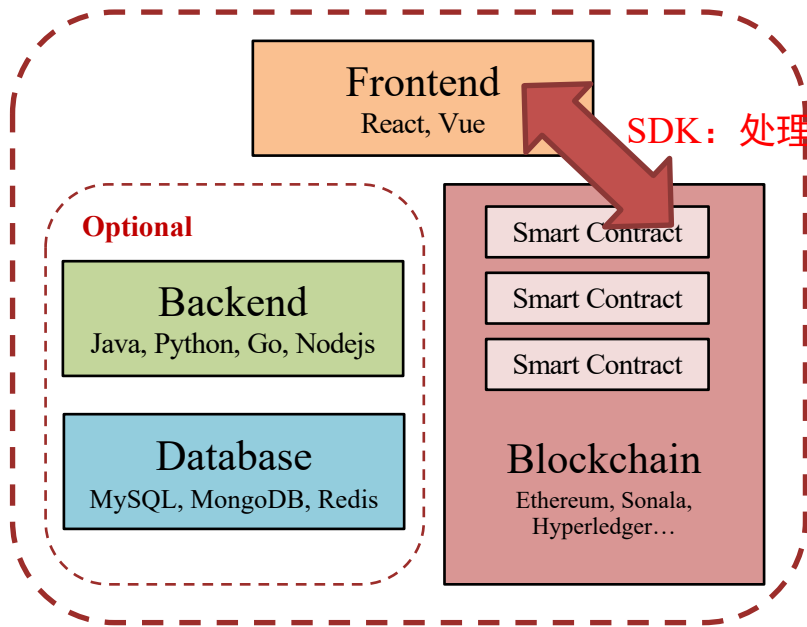


部署合约全过程，来看一个例子/demo-tutorial/contracts

```
contract Calculator {  
  
    function sum(uint256 a, uint256 b) pure external returns (uint256 c) {  
        unchecked {c = a + b;}  
    }  
  
    function safeSumUint8(uint8 a, uint8 b) pure external returns (uint8 c) {  
        unchecked {c = a + b;}  
    }  
  
    function unsafeSumUint8(uint8 a, uint8 b) pure external returns (uint8 c) {  
        c = a + b;  
    }  
  
    function multiply(uint256 a, uint256 b) pure external returns (uint256 c) {  
        unchecked {c = a * b;}  
    }  
  
    function getUserAddressHash() view external returns (bytes32) {  
        // console.log(msg.sender);  
        return keccak256(abi.encodePacked(msg.sender));  
    }  
}
```



- 一. 以太坊开发环境搭建
- 二. 以太坊编程环境搭建
- 三. 与以太坊交互（以太坊SDK）
- 四. DApp开发框架和流程
- 五. DApp优秀案例



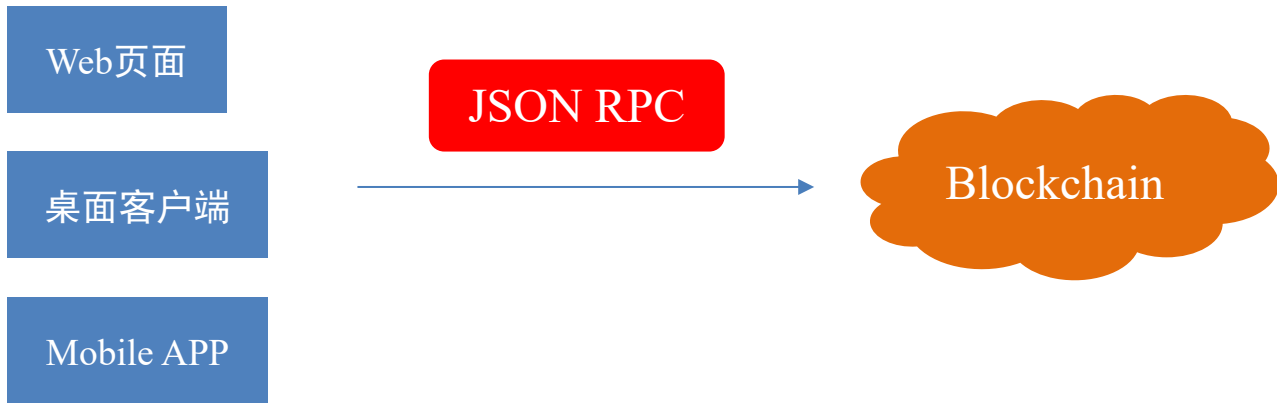
- 前端界面开发（React、Vue）
- 选定一条区块链，编写对应智能合约（以太坊、Solidity）
- 智能合约部署到区块链上
- 前端调用智能合约的方法（SDK）
- （可选）用后端完成其它功能

到目前为止，我们在安装了Solidity编译器，在本地搭建了以太坊私有链，也完成了合约框架的安装，接下来我们需要编写合约，然后部署合约，并在前端和合约交互（调用合约）



JSON RPC

JSON-RPC是一种无状态的，轻量级的远程过程访问协议，并使用JSON作为其数据格式。JSON-RPC主要是在处理过程中定义了一些数据结构和规则，能够在不同的消息传递环境中如Sockets、HTTP中传递信息。





链接：https://github.com/ethereum/wiki/wiki/JSON-RPC#eth_compilesolidity

该网页文档详细介绍了**JSON RPC API**和其一些可调用的接口，其他的某些接口在接下来的讲述中将会用到。如果同学们理解并掌握了**JSON RPC API**里面的内容，你将在区块链开发技术上上了一个台阶。

编译合约是为了获取合约的字节码，用于后面合约的部署。



示例一：获得调用账户

当前以太坊私链中可能有多个创建账户，需要**选定一个账户作为调用部署合约、调用合约方法的发起者**。并且该账户中要包含足够的以太币。这里使用挖矿基地址作为交易的发起者，该地址也就是默认当前账户中的第一个账户。在终端中执行以下命令：

```
curl --data '{"jsonrpc":"2.0","method":"eth_coinbase","id":1}' localhost:7545
```

返回结果：

```
{"jsonrpc":"2.0","id":1,"result":"0x90c2323cdeff75fd82e65ac496fc45eafadf4563"}
```

返回结果中的“**0x90c2323cdeff75fd82e65ac496fc45eafadf4563**”就是要使用的发起交易的账户。



示例二：部署合约

在B终端中执行以下命令，params参数中的from就是发起该笔交易的账户，data参数中就是该合约的字节码，就是我们之前获得的智能合约字节码。

```
curl --data '{"jsonrpc":"2.0","method": "eth_sendTransaction", "params": [{  
  "from": "0x90c2323cdeff75fd82e65ac496fc45eafadf4563",  
  "gas": "0xb8a9",  
  "data":  
    "0x6060604052605f8060106000396000f3606060405260e060020a6000350463c6888fa1811  
4601a575b005b60586004356007810260609081526000907f24abdb5865df5079dcc5ac590ff  
6f01d5c16 edbc5fab4e195d9febd1114503da90602090a15060070290565b5060206060f3"}],  
  "id": 6  
}]' localhost:8545
```



在终端执行上述命令后，**交易需要被挖矿才能确认**。因此终端返回结果中的result是本次交易（部署合约）的交易Hash。

```
{"jsonrpc": "2.0", "id": 6, "result": "0x183685e0ac93e80b0cde97e8b53d425e15bf84d76b073586ad23c66e53085f1f"}
```

根据**交易Hash**查询交易回执后，才能得到当前合约部署成功后的**合约地址**，使用该合约地址调用合约中的方法。

```
curl --data  
'{"jsonrpc": "2.0", "method": "eth_getTransactionReceipt",  
  "params": ["0x183685e0ac93e80b0cde97e8b53d425e15bf84d76b073586ad23c66e53085f1f"],  
  "id": 1}' localhost:8545
```



示例三：调用合约方法

调用合约方法如下，其中to参数中为合约地址，data为调用合约需要传入的参数。data参数中包括了调用合约中的哪个方法和参数值，简称为payload。确认该交易也需要挖矿。

[illegible]

在终端中计算**payload**中的方法选择符对应的字节，选取Keccak散列表的前4个字节，并进行十六进制编码。

```
> web3.sha3("multiply(uint256)").substring(0, 8) "0xc6888f"
```



终端返回结果，通过返回result中的交易Hash我们可以查询本次交易的详情。

```
{"jsonrpc":"2.0","id":8,"result":"0xf3efaa16da4c439eb960387e35468a6ac527f5d9586e7062f69c319364661b"}
```

根据交易Hash查找交易详情

由于一笔交易发生后，真正的返回结果只是交易Hash，可以利用该交易Hash来获得本次交易的真正的详情，以及一些方法的返回值。

在终端中执行以下命令：

```
curl --data
```

```
'{"jsonrpc":"2.0","method":"eth_getTransactionReceipt",  
"params":["0xf3efaa16da4c439eb960387e35468a6ac527f5d9586e7062f69c319364661b"],  
"id":1}' localhost:8545
```

终端返回结果，查看其中的logs.data字段，返回的就是方法真正的返回值。



JSON-RPC



WEB3.JS

<https://web3js.readthedocs.io/en/v3.0.0-rc.5/getting-started.html>



问题：在调用合约方法的时候，怎么知道合约里有哪些方法，应该怎么传参和得到结果？

答案是用ABI文件作为调用合约的标准。

`contract Calculator {`

```
function sum(uint256 a, uint256 b) pure external returns (uint256 c) {
    unchecked { c = a + b; }
}
```

```
function safeSumUint8(uint8 a, uint8 b) pure external returns (uint8 c) {
    unchecked { c = a + b; }
}
```

```
function unsafeSumUint8(uint8 a, uint8 b) pure external returns (uint8 c) {
    c = a + b;
}
```

```
function multiply(uint256 a, uint256 b) pure external returns (uint256 c) {
    unchecked { c = a * b; }
}
```

```
function getAddressHash() view external returns (bytes32) {
    // console.log(msg.sender);
    return keccak256(abi.encodePacked(msg.sender));
}
```

`}`

生成ABI文件：xxxx.json



以multiply方法为例

输入

```
{
  "inputs": [
    {
      "internalType": "uint256",
      "name": "a",
      "type": "uint256"
    },
    {
      "internalType": "uint256",
      "name": "b",
      "type": "uint256"
    }
  ],
  "name": "multiply",
  "outputs": [
    {
      "internalType": "uint256",
      "name": "c",
      "type": "uint256"
    }
  ],
  "stateMutability": "pure",
  "type": "function"
},
```

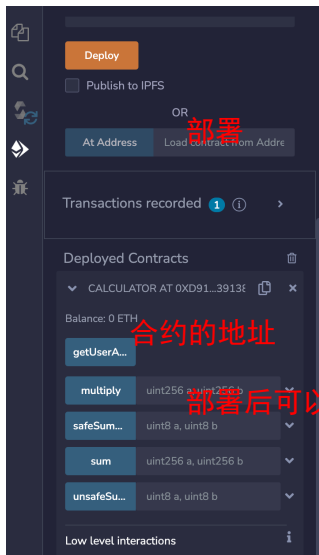
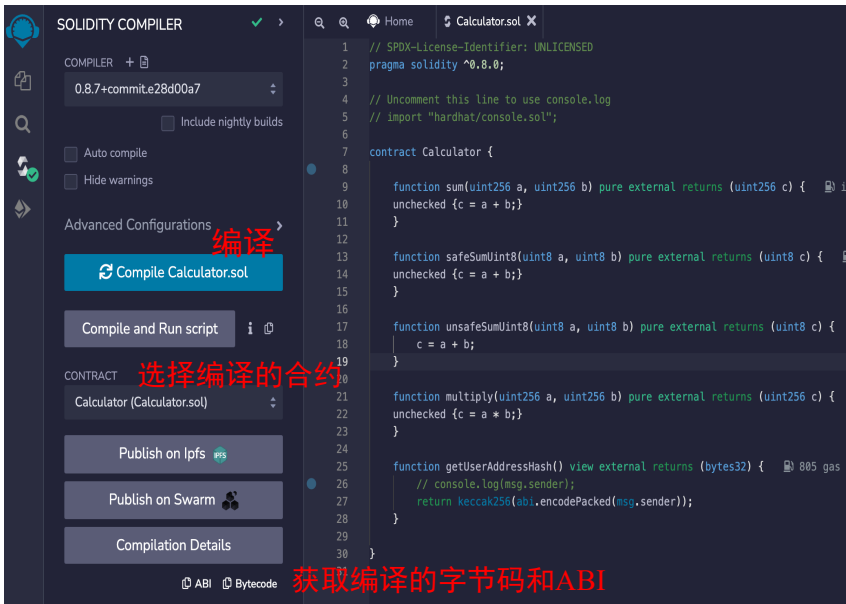
函数名
输出

部署合约全过程，来看一个例子/demo-tutorial/contracts



第一步：在Remix或者本地编译和部署合约

由于web3本身就是使用JavaScript实现的接口，所以下面的终端命令调用也是使用JavaScript实现。Calculator合约代码放入Remix编译，并获得其字节码与ABI：



编译和部署合约后，获得了ABI文件和合约的地址，就可以在本地或者前端调用合约的方法

当然，字节码和ABI也可以在本地编译获得



第一步：在Remix或者本地编译和部署合约

由于web3本身就是使用JavaScript实现的接口，所以下面的终端命令调用也是使用JavaScript实现。Calculator合约代码放入Remix编译，并获得其字节码与ABI：

```
contract Calculator {
```

```
    function sum(uint256 a, uint256 b) pure external returns (uint256 c) {  
        unchecked {c = a + b;}  
    }
```

```
    function safeSumUint8(uint8 a, uint8 b) pure external returns (uint8 c) {  
        unchecked {c = a + b;}  
    }
```

```
    function unsafeSumUint8(uint8 a, uint8 b) pure external returns (uint8 c) {  
        c = a + b;  
    }
```

```
    function multiply(uint256 a, uint256 b) pure external returns (uint256 c) {  
        unchecked {c = a * b;}  
    }
```

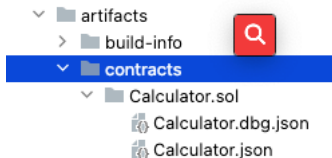
```
    function getUserAddressHash() view external returns (bytes32) {  
        // console.log(msg.sender);  
        return keccak256(abi.encodePacked(msg.sender));  
    }
```

```
}
```

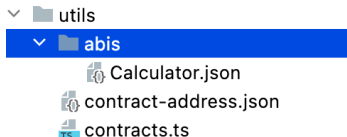
编译合约，得到部署的地址

```
→ contracts npx hardhat run ./scripts/deploy.ts --network ganache  
contract Calculator deployed to 0x7f5568b00d911e8665260d9B235a367F1679429f
```

生成ABI文件：xxxx.json



将生成的ABI和合约地址迁移到前端工程



```
{  
  "inputs": [  
    {  
      "internalType": "uint256",  
      "name": "a",  
      "type": "uint256"  
    },  
    {  
      "internalType": "uint256",  
      "name": "b",  
      "type": "uint256"  
    }  
  ],  
  "name": "multiply",  
  "outputs": [  
    {  
      "internalType": "uint256",  
      "name": "c",  
      "type": "uint256"  
    }  
  ],  
  "stateMutability": "pure",  
  "type": "function"  
},
```



第二步：在前端初始化合约（假设要调用已经部署的合约）

2.0：安装web3.js对象

npm install web3

2.1：初始化web3对象

```
let web3 = new Web3(new Web3.providers.HttpProvider("http://localhost:7545");
```

切换成自己的本地网络，假设此时合约部署在了ganache上

// 修改地址为部署的合约地址

const calculatorAddress = Address.calculator

const calculatorABI = Calculator.abi

2.2：初始化Contract对象

// 获取合约实例

const calculatorContract = new web3.eth.Contract(calculatorABI, calculatorAddress);

```
const calculatorContract = new web3.eth.Contract(calculatorABI, calculatorAddress);
```

2.3：通过合约实例来调用合约方法

```
const result = await calculatorContract.methods.multiply(a, b).call() Call和Send有什么区别？
```

```
await calculatorContract.methods.getUserAddressHash().send({
```

```
  from: accounts[0]
```

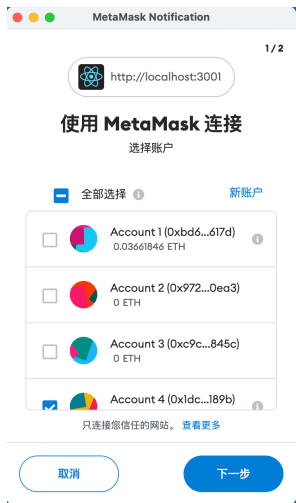
```
})
```

更多用法请参考：<https://web3js.readthedocs.io/en/v3.0.0-rc.5/web3-eth-contract.html>



问题：上述过程中似乎没有涉及到用户的授权，如果我们需要用户在前端授权，如何操作呢？（例如：某个用户向另一个用户transfer Token）

答案：利用钱包（例如Metamask ）。钱包是用户的代理工具，用户将自己的信息存储在浏览器钱包中，App需要授权的时候可以让小狐狸请求用户的签名和授权。 <https://docs.metamask.io>



```
try {  
  // 如果当前小狐狸不在本地链上，切换Metamask到本地测试链  
  if (ethereum.chainId !== GanacheTestChainId) {  
    const chain = {  
      chainId: GanacheTestChainId, // Chain-ID  
      chainName: GanacheTestChainName, // Chain-Name  
      rpcUrls: [GanacheTestChainRpcUrl], // RPC-URL  
    };  
  
    try {  
      // 尝试切换到本地网络  
      await ethereum.request({method: 'wallet_switchEthereumChain', params: [{chainId: chain.chainId}]});  
    } catch (switchError: any) {  
      // 如果本地网络没有添加到Metamask中，添加该网络  
      if (switchError.code === 4902) {  
        await ethereum.request({method: 'wallet_addEthereumChain', params: [chain]});  
      }  
    }  
  }  
  
  // 小狐狸成功切换网络了，接下来让小狐狸请求用户的授权  
  await ethereum.request({method: 'eth_requestAccounts'});  
  // 获取小狐狸拿到的授权用户列表  
  const accounts = await ethereum.request({method: 'eth_accounts'});  
  // 如果用户存在，展示其account，否则显示错误信息  
  setAccount({value: accounts[0] || 'Not able to get accounts'});  
} catch (error: any) {  
  alert(error.message)  
}
```

如何在前端连接小狐狸？来看一个例子：
</demo-tutorial/frontend>



第三步：利用Web3.js使用Metamask提供的Provider，发起交易并获取用户的签名

Metamask在安装后，会在浏览器window对象中注入ethereum对象和web3对象，我们可以直接使用web3对象提供的Provider来构建与区块链的RPC连接。

```
let web3 = new Web3(window.web3.currentProvider)
```

```
// @ts-ignore
// 创建web3实例
// 可以阅读获取更多信息https://docs.metamask.io/guide/provider-migration.html#replacing-window-web3
let web3 = new Web3(window.web3.currentProvider)

// 修改地址为部署的合约地址
const calculatorAddress = Address.calculator
const calculatorABI = Calculator.abi

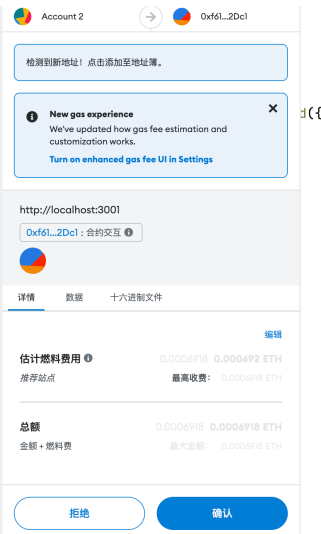
// 获取合约实例
const calculatorContract = new web3.eth.Contract(calculatorABI, calculatorAddress);

// 导出web3实例和其它部署的合约
export {web3, calculatorContract}
```

如何发起交易并获取用户签名授权？来看一个例子：</demo-tutorial/frontend>

```
const onCalcUserAccountHash = async () => {
  if(calculatorContract) {
    const accounts = await web3.eth.getAccounts()
    if(accounts.length === 0) {
      alert('Not connected yet.')
      return
    }
  }
}
```

```
const result = await calculatorContract.methods
  .calcUserAccountHash(accounts[0])
  .send({from: accounts[0]})
// 在这个send方法调用后，合约内部对交易进行了签名
// 返回的结果是一个交易的Hash
console.log(result)
return result
```





Send和Call的区别

```
const result = await calculatorContract.methods.multiply(a, b).call()  
setResult(result)
```

来看一个例子: </frontend/demo-tutorial>

```
const result = await calculatorContract.methods.getUserAddressHash().send({  
  from: accounts[0]  
})
```

// 在这个send方法调用后, 合约内部对交易进行处理, 并打包上链, 结果并不能马上返回。

// 返回的结果是一个交易的Hash。

// 所以, 在需要获取合约数据处理的结果时, 我们需要在合约中使用event-emit模式, 通过事件获取函数处理的结果。

```
console.log(result)
```

1. Call方法只是对数据的读取。可以立即返回结果, 但调用的函数本身需要不改变合约状态, 对应的是合约的修饰符View、Pure。Call方法不需要用户进行签名。
2. Send方法一般用于对数据的修改。无法直接拿到返回值, 调用的函数本身可能对合约的数据和状态进行修改。Send方法需要拿到用户的签名, 从而在函数内使用msg.sender等变量。
3. 为了获取合约函数内数据处理的结果, 我们一般使用event-emit模式来获取结果。



总结一下

1. 我们利用ganache搭建了本地的测试网络。
2. 我们利用hardhat编写合约，测试合约，并将合约部署到ganache的测试网络中。
3. 我们利用web3.js来完成前端和区块链的RPC交互过程，调用合约，获取结果。
4. 为了方便的进行签名授权，我们一般用Metamask这样的钱包管理自己的账户。
5. 在构建DApp时，为了方便的授权，**需要在前端允许用户连接我们的应用**，并构建逻辑，在需要时候让用户完成授权工作。



- 一. 以太坊开发环境搭建
- 二. 以太坊编程环境搭建
- 三. 与以太坊交互（以太坊SDK）
- 四. DApp开发框架和流程
- 五. DApp优秀案例



例子：/demo-lottery-application/contracts

1. 合约编写和部署

首先我们完成合约逻辑的编写，并将其部署在主网/测试网/本地网络上。

新建一个hardhat合约工程：

```
→ demo-Token-application git:(master) × rm -rf contracts
→ demo-Token-application git:(master) × mkdir contracts
→ demo-Token-application git:(master) × cd contracts
→ contracts git:(master) × npm init --yes
```

```
→ contracts git:(master) × npx hardhat
888 888 888 888 888
888 888 888 888 888
888 888 888 888 888
8888888888 8888b. 888d888 .d88888 88888b. 8888b. 888888
888 888 "88b 888P" d88" 888 888 "88b "88b 888
888 888 .d888888 888 888 888 888 .d888888 888
888 888 888 888 888 Y88b 888 888 888 888 Y88b.
888 888 "Y888888 888 "Y888888 888 888 "Y888888 "Y888
```

👉 Welcome to Hardhat v2.11.2 👉

```
✓ What do you want to do? · Create a TypeScript project
✓ Hardhat project root: · /Users/hinsliu/Desktop/区块链与数字货币-助教/demo/demo-Token-application/contracts
✓ Do you want to add a .gitignore? (Y/n) · n
✓ Do you want to install this sample project's dependencies with npm (@nomicfoundation/hardhat-toolbox)? (Y/n) · y
```

```

├─ demo-Token-application
├─ contracts
├─ contracts
├─ Lock.sol
├─ node_modules library root
├─ scripts
├─ deploy.ts
├─ test
├─ Lock.ts
├─ hardhat.config.ts
├─ package.json
├─ package-lock.json
├─ README.md
├─ tsconfig.json
```



- **合约编写** 这里我们编写一个彩票合约，基于我们发行的Token完成彩票购买和开奖的逻辑

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.0;

import "./MyERC20.sol";

contract Lottery {

    uint256 constant public PLAY_AMOUNT = 500;
    address public manager; // 管理员, 用来开奖
    address[] public players; // 玩家
    address public winner; // 胜者
    MyERC20 public myERC20; // 彩票相关的代币合约

    modifier onlyManager {
        require(msg.sender == manager);
    }

    // 管理员
    constructor() {
        myERC20 = new MyERC20("ZJUToken", "ZJUTokenSymbol");
        manager = msg.sender;
    }

    // 投注
    function play() public {
        // 验证转账条件
        myERC20.transferFrom(msg.sender, address(this), PLAY_AMOUNT);
        // 把参与量加入到彩票池中
        players.push(msg.sender);
    }

    // 从参与玩家中随机抽取一个玩家, 这里使用哈希算法来随机: Hash(当前时间, 区块的挖矿难度, 彩票数量)
    function draw() onlyManager public {
        require(players.length != 0, "No player involved in the game");
        bytes32 hash = keccak256(abi.encodePacked(block.timestamp, block.difficulty, players.length));
        uint256 hashValue = uint256(hash);

        // 随机抽到一个玩家
        uint256 index = hashValue % players.length;
        winner = players[index];

        // 将本合约的所有Token发还给赢家
        myERC20.transfer(winner, myERC20.balanceOf(address(this)));
        // 清理彩票池
        delete players;
    }

    // 退款
    function refund() onlyManager public {
        // 因为参与的玩家退款
        for(uint256 i = 0; i < players.length; i++){
            myERC20.transfer(players[i], PLAY_AMOUNT);
        }
        // 清理彩票池
        delete players;
    }
}
```

通过modifier限制一些功能只有管理者可以调用

新建一个代币合约

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.0;

import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

contract MyERC20 is ERC20 {

    mapping(address => bool) claimedAirdropPlayerList;

    constructor(string memory name, string memory symbol) ERC20(name, symbol) {

        function airdrop() external {
            require(claimedAirdropPlayerList[msg.sender] == false, "This user has claimed airdrop already");
            _mint(msg.sender, 10000);
            claimedAirdropPlayerList[msg.sender] = true;
        }
    }
}
```

这里引用了OpenZeppelin的标准ERC20合约实现

空投

实现了如下功能：

- 用户领取空投，参与彩票活动
- 用户花费一定的代币，参与彩票
- 管理员开奖，随机选择玩家，获得所有奖金
- 管理员退款

- **合约部署** 新建合约环境，将合约部署到ganache上，获得合约地址和ABI

测试的步骤这里略过

编译合约

```
+ contracts git:(master) × npx hardhat compile
Generating typings for: 1 artifacts in dir: typechain-types for target: ethers-v5
Successfully generated 18 typings!
Compiled 1 Solidity file successfully
```

新建ganache测试环境并启动

PORT NUMBER

8545

WORKSPACE

WORKSPACE NAME

Blockchain-Course-Lottery-Demo

A friendly name for this workspace.

TRUFFLE PROJECTS

Link Truffle projects to this workspace by adding their truffle-config.js or truffle.js file to this workspace.

This will show useful contract and event data to better understand what's going on under the hood.

ADD PROJECT

REMOVE PROJECT

将ganache的账户信息和网络信息写到hardhat配置文件中

```
const config: HardhatUserConfig = {
  // Solidity compiler version
  solidity: "0.8.9",
  networks: {
    ganache: {
      // Change the url according to your ganache configuration
      url: 'http://localhost:8545',
      // Change these accounts private keys according to your ganache configuration.
      accounts: [
        'e7b3551781108825f89f9bfc50d03c61f3e8f0169494b6b4ae8594e74a5d3ec',
        '4b2f81d42c979d50b3019bc3a8ca1524b4342c00912db8c3dda08db78c3fd390',
        'a656b929893a0133cced4f942670f7263b7a229d5a887bec25358acb99a951d5',
        'c037f223098073b66d6fc1d67c258e9c800b74d14fa600a6baf8c02178ff5d16',
      ],
    },
  },
  paths: {
    sources: "./contracts",
    tests: "./test",
    cache: "./cache",
    artifacts: "./artifacts"
  },
};

export default config;
```

部署合约拿到地址

```
+ contracts git:(master) × npx hardhat run ./scripts/deploy.ts --network ganache
Compiled 1 Solidity file successfully
lottery contract has been deployed successfully in 0x98c453bd60C9c7591972677B724055D21E4A9Ac5
erc20 contract has been deployed successfully in 0x39E5b0Efe723F39a6A4fDB8B018b62CDa321E52b
```



例子：/demo-lottery-application/lottery-frontend

2. 前端开发：以React为例

React是当前最流行的Web前端框架，是用于构建用户界面的 JavaScript 库，可以非常方便的集成以太坊的web3.js接口。

目前大部分DApp应用是基于React框架来开发的，下面来看如何安装React、加载以太坊web3模块、调用web3.js接口、部署DApp应用。

编写前端应用

导入web3模块

调用web3接口

部署DApp

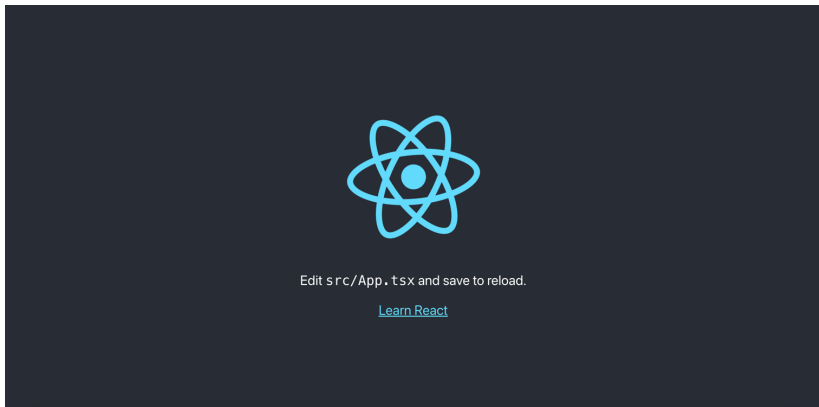




- **安装React**

这里我们使用React官方提供的[create-react-app](https://create-react-app.dev/)工具，新建一个TypeScript项目

```
npx create-react-app lottery-frontend --template typescript  
cd demo-frontend  
npm run start
```



当浏览器出现左侧页面
时，表示初始化成功！

在浏览器中输入<http://localhost:3000> 来访问应用。



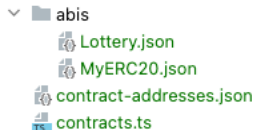
• 编写前端页面样式和代码

React项目创建完成后，在src下编写我们的页面源代码。由于Dapp通常是Serverless应用，因此我们只需要在前端使用web3.js与以太坊进行交互接口。这里略过细节，得到以下简单的展示页面。

安装 web3

```
npm install -D web3
```

编写读取ABI文件和合约地址的逻辑，实例化合约



```
const Web3 = require('web3');

// @ts-ignore
// 创建web3实例
// 可以阅读获取更多信息https://docs.metamask.io/guide/provider-migration.html#replacing-window-web3
let web3 = new Web3(window.web3.currentProvider)

// 修改地址为部署的合约地址
const lotteryAddress = Addresses.lottery
const lotteryABI = Lottery.abi
const myERC20Address = Addresses.myERC20
const myERC20ABI = MyERC20.abi

// 获取合约实例
const lotteryContract = new web3.eth.Contract(lotteryABI, lotteryAddress);
const myERC20Contract = new web3.eth.Contract(myERC20ABI, myERC20Address);

// 导出web3实例和其它部署的合约
export {web3, lotteryContract, myERC20Contract}
```



- **编写与区块链交互的逻辑**

在src/App.js中使用web3编写与区块链交互的代码逻辑

看代码！

在浏览器中输入http://localhost:3000 来访问DApp应用。

将ganache中账户导入小狐狸MetaMask，进行测试和体验



浙大彩票DEMO

领取浙大币空投

管理员地址：0xFdE66Aa1b06cb7cf8c2447B7a0e91dcB0953D5F1

当前用户：0x58541A2a5dA50b7B16F6F739f099F11A28032686

当前用户拥有浙大币数量：10500

花费500浙大币，赢取更多浙大币！每个人可以参与多次，参与越多，得奖概率越高！

⚠ 已有0人/次参加

奖池有金额 0 个ZJUToken

操作栏

投注产生希望

开奖（仅限管理员使用）

退款（仅限管理员使用）



- 一. 以太坊开发环境搭建
- 二. 以太坊编程环境搭建
- 三. 与以太坊交互（以太坊SDK）
- 四. DApp开发框架和流程
- 五. DApp优秀案例



Example: ERC20

Etherscan

Eth: \$546.01 (-3.80%) | 35 Gwei

All Filters Search by Address / Txn Hash / Block / Token / Ens

HomeBlockchainTokensResourcesMoreSign In

BuyExchangeEarnGaming

Token ChainLink Token

Overview [ERC-20]

PRICE
\$11.7800 @ 0.021575 Eth (-5.31%)

FULLY DILUTED MARKET CAP ⓘ
\$11,780,000,000.00

Total Supply:

1,000,000,000 LINK ⓘ

Holders:

289,402 addresses

Transfers:

6,181,453

Profile Summary [Edit]

Contract:

0x514910771af9ca656af840df83e8264ecf986ca

Decimals:

18

Official Site:

<https://chain.link/>

Social Profiles:

Transfers Holders Info Exchange DEX Trades Read Contract Write Contract Analytics Comments

A total of 6,181,453 transactions found
(Showing the last 100k records)

First

<

Page 1 of 4000

>

Last

	Txn Hash	Age	From	To	Quantity
	0x400efdf94015110ead...	1 min ago	0x7c5609cfa86eb533...	0x4861cce330215205e...	40
	0x1488f259c9bca319e...	1 min ago	0xf0bc4c1cbc24642a1...	0xb73cea4f54518bcc9...	1
	0x2a44b710e46923a8e...	1 min ago	0x2045c1b9c01a66ee6...	0xe8025e29af3e2ad98...	8.49838616
	0xd7175303dae6c611...	1 min ago	0xb171ed597b21c96a1...	0x49051b6fd9113bde5...	20

This website uses cookies to improve your experience and has an updated Privacy Policy. [Got It](#)



Example: ERC20

```
1 contract ERC20 {
2     //*****9个函数*****
3     //1.代币的名字, 如: "黑马币"
4     function name() constant public returns (string name);
5     //2.代币的简称, 例如: HMB
6     function symbol() public constant returns (string symbol);
7     //3.代币的最小分割量 token使用的小数点后几位。比如如果设置为3, 就是支持0.001表示
8     function decimals() public constant returns (uint8 decimals);
9     //4.token的总量
10    function totalSupply() public constant returns (uint totalSupply);
11
12    //5.余额 返回某个地址(账户)的账户余额
13    function balanceOf(address _owner) public constant returns (uint balance);
14
15    /*6.转账 交易代币 从消息发送者账户中往_to账户转数量为_value的token,
16       从代币合约的调用者地址上转移_value的数量token到地址_to
17       【注意: 并且必须触发Transfer事件】*/
18    function transfer(address _to, uint _value) public returns (bool success);
19
20    /*7.两个地址转账
21       从账户_from中往账户_to转数量为_value的token, 与approve方法配合使用
22       从地址_from发送数量为_value的token到地址_to
23       【注意: 并且必须触发Transfer事件】
24       transferFrom方法用于允许合约代理某人转移token。条件是from账户必须经过了approve。*/
25    function transferFrom(address _from, address _to, uint _value) public returns (bool success);
26
27    //8.批准_spender能从合约调用账户中转出数量为_value的token
28    function approve(address _spender, uint _value) public returns (bool success);
29    //9.获取_spender可以从账户_owner中转出token的剩余数量
30    function allowance(address _owner, address _spender) public constant returns (uint remaining);
31
32    //*****2个事件*****
33    //1.发生转账时必须触发的事件,transfer 和 transferFrom 成功执行时必须触发的事件
34    event Transfer(address indexed _from, address indexed _to, uint _value);
35    //2.当函数 approve(address _spender, uint256 _value)成功执行时必须触发的事件
36    event Approval(address indexed _owner, address indexed _spender, uint _value);
37 }
```



Example: ERC20 一键发币

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity ^0.6.2;
3
4 import "https://github.com/OpenZeppelin/openzeppelin-contracts/blob/v3.1.0/contracts/token/ERC20/ERC20.sol";
5
6 contract Token is ERC20 {
7
8     constructor(uint256 _initialSupply) public ERC20("EVAToken", "EVA") {
9         _mint(msg.sender, _initialSupply);
10    }
11 }
```

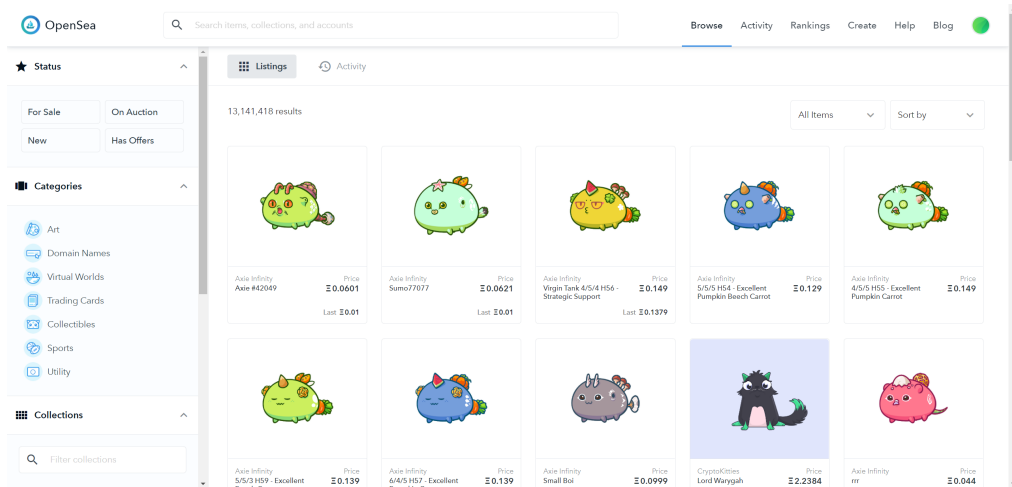


Example: ERC721

ERC721 简要解释是 “Non-Fungible Tokens”，翻译为不可拆分的Token, 英文简称为 “NFT”，简单理解为每个Token都是独一无二的。也就是说ERC721的每个Token都拥有独立唯一的token_id编号，token_id只能被一个owner(i.e. eth address) 所拥有；



CryptoKitties





Example: ERC721

ERC721 最为一个合约标准，提供了在实现 ERC721 代币时必须要遵守的协议，要求每个 ERC721 标准合约需要实现 ERC721 及 ERC165 接口，接口定义如下：

```
1  pragma solidity ^0.6.0;
2
3  interface ERC721 /* is ERC165 */ {
4
5      event Transfer(address indexed _from, address indexed _to, uint256 _tokenId);
6      event Approval(address indexed _owner, address indexed _approved, uint256 _tokenId);
7      event ApprovalForAll(address indexed _owner, address indexed _operator, bool _approved);
8
9      function balanceOf(address _owner) external view returns (uint256);
10     function ownerOf(uint256 _tokenId) external view returns (address);
11
12     function safeTransferFrom(address _from, address _to, uint256 _tokenId, bytes data) external payable;
13     function safeTransferFrom(address _from, address _to, uint256 _tokenId) external payable;
14     function transferFrom(address _from, address _to, uint256 _tokenId) external payable;
15
16     function approve(address _approved, uint256 _tokenId) external payable;
17     function setApprovalForAll(address _operator, bool _approved) external;
18     function getApproved(uint256 _tokenId) external view returns (address);
19     function isApprovedForAll(address _owner, address _operator) external view returns (bool);
20 }
```

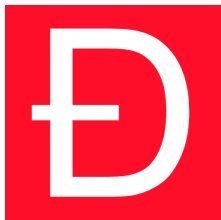


Example: ERC721 – 一键NFT

```
1 // contracts/GameItem.sol
2 // SPDX-License-Identifier: MIT
3 pragma solidity ^0.6.0;
4
5 import "https://github.com/OpenZeppelin/openzeppelin-contracts/blob/v3.1.0/contracts/token/ERC721/ERC721.sol";
6 import "https://github.com/OpenZeppelin/openzeppelin-contracts/blob/v3.1.0/contracts/utils/Counters.sol";
7
8 contract GameItem is ERC721 {
9     using Counters for Counters.Counter;
10    Counters.Counter private _tokenIds;
11
12    constructor() public ERC721("GameItem", "ITM") {}
13
14    function awardItem(address player, string memory tokenURI)
15        public
16        returns (uint256)
17    {
18        _tokenIds.increment();
19
20        uint256 newItemId = _tokenIds.current();
21        _mint(player, newItemId);
22        _setTokenURI(newItemId, tokenURI);
23
24        return newItemId;
25    }
26 }
```



CryptoKitties



THE DAO



Uniswap

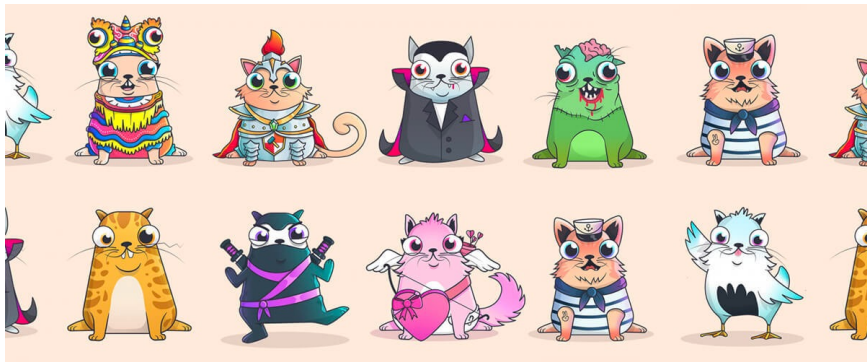


CryptoKitties

<https://www.cryptokitties.co>

加密猫(CryptoKitties)是全球首款基于区块链的数字宠物养成游戏，你可以领养、繁殖猫咪。每只猫拥有独特的基因，并在链上是唯一的数字资产。你也可以把它们当做传统收藏品那样交易，区块技术可以保证交易安全进行。

与以往收藏品不同，你还可以用两只猫咪繁殖一些拥有独特基因的崭新后代。





合约

<https://github.com/dapperlabs/cryptokitties-bounty/tree/master/contracts>

- KittyCore: 主合约, 承载主要逻辑
- KittyAccessControl: 管理控制权限
- KittyBase: 猫咪的基础数据
- KittyOwnership: 基于ERC721标准, 猫咪Token
- KittyBreeding: 猫咪繁殖
- KittyAuctions: 猫咪交易
- KittyMinting: 0代猫咪工厂



合约 KittyAccessControl

包含了拥有特殊权限的账户地址以及每个地址对应的权限内容。

内置pause和unpause方法，可以暂停和继续合约执行。当暂停时用户将不能买卖和繁殖猫咪（这与去中心化的理念有些许违背）

```
contract KittyAccessControl {

    address public ceoAddress;
    address public cfoAddress;
    address public cooAddress;

    modifier onlyCLevel() {
        require(
            msg.sender == cooAddress ||
            msg.sender == ceoAddress ||
            msg.sender == cfoAddress
        );
    }

    // 三个角色中任意一个都可以暂停合约
    function pause() external onlyCLevel whenNotPaused {
        paused = true;
    }

    // 只有COO
    function unpause() public onlyCEO whenPaused {
        // can't unpause if contract was upgraded
        paused = false;
    }
}
```



合约 KittyBase

该合约定义了猫咪的各种属性、出生日期、父代ID等信息，以及猫咪相关的行为事件

合约方法

- `_transfer`: 猫咪转移
- `_createKitty`: 生成新的猫咪
- `setSecondPerBlock`: 修改区块间隔时间

核心数据结构

```
struct Kitty {  
    uint256 genes; // 猫咪的基因，决定了猫咪的各种特征  
    uint64 birthTime; // 猫咪出生的日期
```

// 猫咪可以再次进行繁殖的最小区块。一只刚出生的猫咪cooldownEndBlock=0，当猫咪需要生产时，需要先判断cooldownEndBlock，只有cooldownEndBlock小于等于当前区块编号时才能繁殖

```
    uint64 cooldownEndBlock;
```

```
    uint32 matronId; // 猫妈妈的ID
```

```
    uint32 sireId; // 猫爸爸的ID
```

// 未繁殖的猫咪siringWithId=0，繁殖后的猫咪siringWithId=父代ID

```
    uint32 siringWithId;
```

// 繁殖冷却时间，随着繁殖次数的增加，冷却时间也会增加，初始冷却时间为：
`_generation/2`，对应的具体冷却时间可以查看cooldowns变量

```
    uint16 cooldownIndex; // 繁殖冷却时间
```

```
    uint16 generation; // 猫咪的代数
```

```
}
```



合约 KittyOwnerShip

保证这里的每一只猫都是独一无二无法互换的（non-fungible）。

ERC-721是以太坊社区的非同质化代币标准，适用于任何无法互换的电子收藏品，每一个Token具有唯一的标识符，常用于表征具有唯一性的数字资产，如票据、房产、稀有藏品等。

ERC-721

<http://erc721.org/>

```
/// @title Interface for contracts conforming to ERC-721: Non-Fungible Tokens ///
contract ERC721 {
    // Events event Transfer(address from, address to, uint256 tokenId);
    event Transfer(address from, address to, uint256 tokenId);
    event Approval(address owner, address approved, uint256 tokenId);

    // Required methods
    function totalSupply() public view returns (uint256 total);
    function balanceOf(address _owner) public view returns (uint256 balance);
    function ownerOf(uint256 _tokenId) external view returns (address owner);
    function approve(address _to, uint256 _tokenId) external;
    function transfer(address _to, uint256 _tokenId) external;
    function transferFrom(address _from, address _to, uint256 _tokenId) external;

    // ERC-165 Compatibility (https://github.com/ethereum/EIPs/issues/165)
    function supportsInterface(bytes4 _interfaceID) external view returns (bool);
}
```



合约 KittyBreeding

合约定义了：

- 提供繁殖服务的公猫列表（之所以说公猫提供服务，是因为默认新出生的猫归母猫的主人所有）
- 提供繁殖新的猫咪的合约方法
- 一些工具方法

该合约依赖于一个定义了基因重组细节的非开源合约 geneScience。

```
function _breedWith(uint256 _matronId, uint256 _sireId) internal {
    // 从猫咪集合中根据id获取相应猫咪
    Kitty storage sire = kitties[_sireId];
    Kitty storage matron = kitties[_matronId];

    // 母猫设置为已怀孕，被记录公猫的id
    matron.siringWithId = uint32(_sireId);

    // 冻结两只猫咪
    _triggerCooldown(sire);
    _triggerCooldown(matron);

    // 清除两只猫的繁殖功能
    delete sireAllowedToAddress[_matronId];
    delete sireAllowedToAddress[_sireId];

    // 每当有猫咪怀孕，计数器+1
    pregnantKitties++;

    // 发送一个怀孕的事件
    Pregnant(kittyIndexToOwner[_matronId], _matronId, _sireId, matron.cooldownEndBlock);
}
```



合约 KittyMinting

负责0世代猫咪的生成。允许我们生成最多5000只推广用的免费赠送0世代猫，而其余的0世代猫则需要通过程序生成（包括决定初始价格）并且直接放置到市场上进行拍卖。

以太坊中的0世代猫上限为50000只，其余的新猫只能通过既存猫咪间“繁殖”来生成了。

```
uint256 public constant PROMO_CREATION_LIMIT = 5000;  
uint256 public constant GEN0_CREATION_LIMIT = 45000;
```

```
function createPromoKitty(uint256 _genes, address _owner) external onlyCOO {  
    address kittyOwner = _owner;  
    if (kittyOwner == address(0)) { kittyOwner = cooAddress; }  
    require(promoCreatedCount < PROMO_CREATION_LIMIT);  
    promoCreatedCount++;  
    _createKitty(0, 0, 0, _genes, kittyOwner);  
}
```

```
function createGen0Auction(uint256 _genes) external onlyCOO {  
    require(gen0CreatedCount < GEN0_CREATION_LIMIT);  
    uint256 kittyId = _createKitty(0, 0, 0, _genes, address(this));  
    _approve(kittyId, saleAuction);  
    saleAuction.createAuction(  
        kittyId,  
        _computeNextGen0Price(),  
        0,  
        GEN0_AUCTION_DURATION, address(this)  
    );  
    gen0CreatedCount++;  
}
```



THE DAO

DAO（Decentralized Autonomous Organization）是一种通过智能合约将个体与个体、个人与组织、或组织与组织联系在一起的新型组织形式。

The DAO项目于2016年4月30日开始，融资窗口开放了28天。截止5月15日这个项目筹得了超过一亿美元，而到整个融资期结束，共有超过11,000位热情的成员参与进来，筹得1.5亿美元，成为历史上最大的众筹项目。The DAO所集资的钱远远超过其创建者的预期。

运作方式：

1. 部署运行智能合约代码
2. 初始融资阶段，人们添加资金来购买DAO代币，来代表其所有权——这个过程叫做众销，或者首次代币发行（ICO）——为其提供所需资源。
3. 当融资结束后，DAO项目就可以开始利用融到的钱真正开始运作。
4. 人们开始像DAO系统管理者提出如何使用这笔钱的方案，并且购买DAO的成员就有资格对这些提案进行投票。
5. 项目收益分红
6. 选择撤资：splitDAO, childDAO



THE DAO

Race-to-Empty攻击：用一种攻击方法，通过多次快速的调用某个代码致使被攻击者的账户中 useful 资产为空的情况。

THE DAO: 不断递归而进行的无限制转账问题

splitDAO func

```
// Burn DAO Tokens
function splitDAO {
  Transfer(msg.sender, 0, balances[msg.sender]);
  // be nice, and get his rewards
  withdrawRewardFor(msg.sender);
  totalSupply -= balances[msg.sender];
  balances[msg.sender] = 0;
  paidOut[msg.sender] = 0;
  return true;
}
```

withdrawRewardFor func

```
function withdrawRewardFor(address _account) noEther internal
returns (bool _success) {
  if ((balanceOf(_account) * rewardAccount.accumulatedInput())
  / totalSupply < paidOut[_account]) { throw; }

  uint reward = (balanceOf(_account) *
  rewardAccount.accumulatedInput()) / totalSupply -
  paidOut[_account];

  // 这里有限递归调用的漏洞，因为提款值的更改在跨合约调用之后
  if (!rewardAccount.payOut(_account, reward)) { throw; }
  paidOut[_account] += reward;
  return true;
}
```



THE DAO

Race-to-Empty攻击：用一种攻击方法，通过多次快速的调用某个代码致使被攻击者的账户中无用资产为空的情况。

THE DAO: 不断递归而进行的无限制转账问题

payOut func

```
function payOut(address _recipient, uint _amount) returns (bool) {
    if (msg.sender != owner || msg.value > 0 || (payOwnerOnly &&
        _recipient != owner)) { throw; }

    if (_recipient.call.value(_amount)()) { //注意这一行，会触发
        _recipient的fallback
        PayOut(_recipient, _amount);
        return true;
    } else {
        return false;
    }
}
```

黑客合约的fallback函数

```
function () {
    vulnerableContract v;
    uint times;
    if (times == 0 && attackModelsOn) {
        times = 1;
        v.splitDAO(); // 继续调用splitDAO开始新一轮提款
    } else {
        times = 0;
    }
}
```



THE DAO

被攻击原因

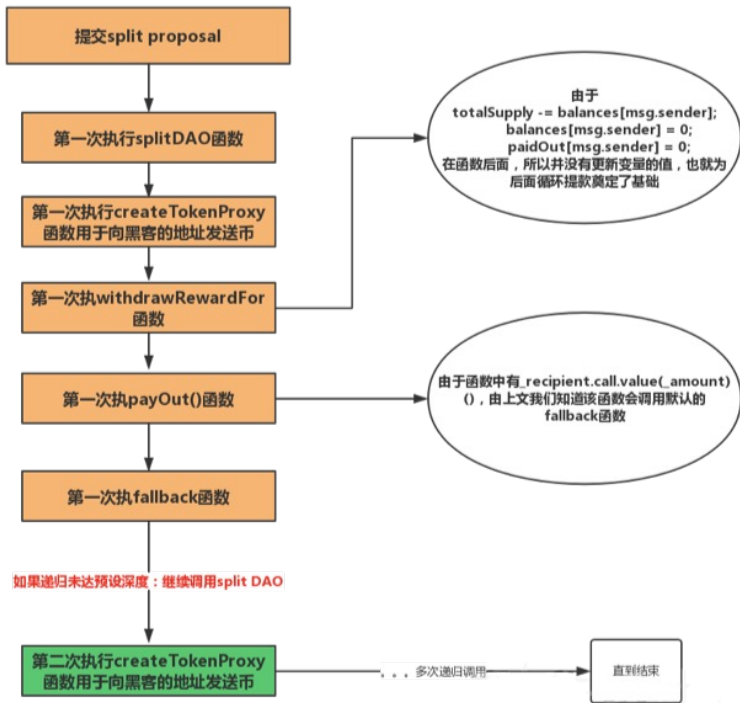
- THE DAO中余额扣除和转账顺序有误
- 未知代码被使用
- 资金存储在一个账户

解决方案

- 2016年6月18日，白帽将2/3未被盗取的资金转移到安全账户；以太坊开启交易验证
- 2016年6月24日，以太坊提交了软分叉方案，阻止提取资金，争取时间
- 2016年6月28日，Felix Lange提出软分叉存在风险
- 2016年6月30日，Vitalik Buterin提出硬分叉设想
- 2016年7月15日，硬分叉方案公布，7月21最终期限
- 2016年7月21日，超过85%的算力支持硬分叉，硬分叉成功



Dapp优秀案例



黑客创建带有
fallback函数的合约

fallback函数在
recipient.call.value()时自动执行

递归未达到深度，
继续调用**splitDAO**



浙江大学
ZHEJIANG UNIVERSITY

Dapp优秀案例



THE DAO

<https://github.com/slockit/DAO>



自动化做市商

- Uniswap 是一个基于以太坊的自动代币交换协议，它的设计目标是：易用性、gas 高利用率、抗审查性和零抽租。
- ease-of-use（易用性）：Token A 换 Token B，在 Uniswap 也只要发出一笔交易就能完成兑换，在其它交易所中可能需要发两笔交易：第一笔将 Token A 换成某种媒介货币，如 ETH, DAI 等，然后再发第二笔交易换成 Token B。
- gas efficiency（gas 高利用率）：在 Uniswap 上消耗的 gas 量是以太坊上的几家主流去中心化交易所中最低的，也就代表在 Uniswap 交易要付的矿工费最少。
- censorship resistance（抗审查性）：抗审查性体现在 Uniswap 上架新 Token 没有门槛，任何人都能在 Uniswap 上架任何 Token。这在去中心化交易所中很少见，虽然大多数的去中心化交易所不会像中心化交易所那样向你收取上市费，但还是需要提交上市申请，通过审查后运营团队才会让你的 Token 可以在他们的交易所上交易。下面是各去中心化交易所上市规则的详情：



传统的交易所有一个订单簿(Order Book), 订单簿上记录着买卖方向, 数量和出价, 交易所负责对买卖双方进行配对, 一旦订单簿中最高价格低于或等于最低价格, 就会促成交易, 同时会产生一个新的成交价, 传统交易所具有以下特点:

- 市场上必须要有用户进行挂单, 要有一定量的订单 (市场流动)。
- 订单必须重叠才能成交, 即买价高于或等于卖价。
- 需要将资产存储在交易所。

在订单簿模型市场中, 买家期望用最低的价格买到想要的资产, 而卖家则是期望用最高价格卖出同一项资产, 如果交易要成立, 买卖双方必须要在价格上达成共识, 一是买家提高出价, 二是卖家降低出价, 如果双方都不要改变出价, 这时候就要依靠做市商的参与, 简单来说, 做市商是一个促进交易的实体, 它会在买卖两个方向上挂单, 让想要交易的参与方只要跟做市商的订单撮合就能完成交易, 而不需要等待对手方出现才能交易, 极大的提高了市场的流动性。





为什么 Uniswap 不采用订单簿模型？

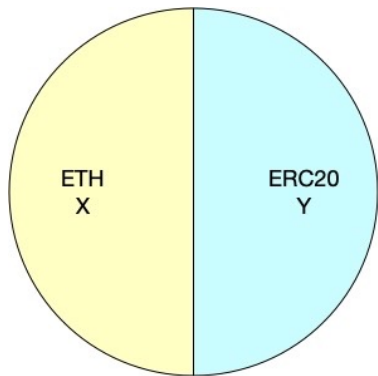
- Uniswap 部署在以太坊上，而以太坊每秒可以出来 15 笔左右的交易，这对于订单簿交易所来不可行，主要原因是：“订单簿模型依赖一个或多个外部做市商对某项资产一直不断的做市，而以太坊的 TPS 过低不支持做市商高频的交易，如果缺少了做市商，那么交易所的流动性立刻会降低，对于用户来说这样的交易所体验很差。”
- Uniswap 采用流动池加恒定乘法公式这种自动化做市商（AMM）模式实现了资产的交换，自动化做市商模式方式不需要买卖双方进行挂单，也不要买卖双方的订单重叠，可以进行自由买卖。
 - 流动池：使用流动池来提供买卖双方交易，做市商只要把资金放入流动池即可
 - 恒定乘法公式：按照流动池中 Token 的数量，自动计算买卖价格



Dapp优秀案例 —— Uniswap



流动池：Uniswap 是无法自己变出钱来，因此需要依赖外部资金向合约中提供流动性，向 Uniswap 流动池中提供流动性的用户被称为流动性提供者，当流动性提供者向 Uniswap 中注入流动性的时候，Uniswap 会铸造出一个流动性代币(LP)，铸出 LP 代币数量是与用户注入的资金所占流动池中的资金比例相关，流动性提供者可以选择在任何时间销毁自己持有的流动性代币。为了鼓励用于向 Uniswap 的流动池中提供更多的流动性，Uniswap 会从每笔交易总额中抽取 0.3% 当成交易手续费，并将手续费全额交给那些将资金注入到 Uniswap 资金池提供流动性的流动性提供者。





核心思想：

- 假设在 Uniswap 中存在一个 ETH-DAI 交易对的流动池，用户在使用 DAI 与 ETH 兑换时需要一个方法来决定交易价格。
- Uniswap 定价模型非常简洁，它的核心思想是一个恒定乘积公式 $x \times y = k$ 。其中 x 和 y 分别代表流动池中两种资产的数量， k 是两种资产数量的乘积。
- 假设乘积 k 是一个固定不变的常量，当用户使用 x 资产从流动池中兑换 y 资产时，流动池中 x 资产的数量会增加， y 资产的数量会减少。由于 k 是恒定的，所以当 x 增长 Δx 时，需要将 y 减少 Δy 才能保持等式的恒定。



交易价格计算：

- InputPrice: 向流动池中放入 ΔX 个 Token 可以兑换出多少个 ΔY Token
- OutputPrice: 从流动池中取出 ΔY 个 Token, 需要向流动池中放入多少个 ΔX Token

$$x' = x + \Delta x = (1 + \alpha)x = \frac{1}{1-\beta}x$$

$$y' = y - \Delta y = \frac{1}{1+\alpha}y = (1 - \beta)y$$

$$xy = x'y' = k$$

$$\text{其中: } \alpha = \frac{\Delta x}{x}, \beta = \frac{\Delta y}{y}$$

$$\Delta x = \alpha x$$

$$x' = (1 + \alpha)x = \frac{1}{1-\beta}x \implies 1 + \alpha = \frac{1}{1-\beta} \implies \alpha = \frac{\beta}{1-\beta}$$

$$\Delta x = \frac{\beta}{1-\beta}x$$

$$\Delta y = \frac{\alpha}{1-\alpha}y$$



交易滑点：

- 交易滑点是指：“在买卖Token时，实际支付的价格和期望的交易价格之间的差别，称为交易滑点”。
- 从流动池中购买 ΔY Token 需要支付 ΔX ，所以本次购买的价格为： $\Delta Y / \Delta X$

$$P = \frac{\Delta X}{\Delta Y} = \frac{\frac{\alpha x}{1+\alpha} y}{y} = (1 + \alpha) \frac{x}{y}$$

$$P' = \frac{x}{y}$$

$$\text{滑点} = P - P' = (1 + \alpha)P' - P' = \alpha P'$$

其中： $P' = \frac{x}{y}$ 表示当前流动池中的价格

从公式可以看出交易的滑点是与 $\alpha = \frac{\Delta X}{x}$ 成线性关系。



流动代币价：

- 当从流动池中购买 Δy 代币后一定会引起当前流动池中的价格波动：其中：P是购买代币后流动池中的币价

$$P = \frac{x'}{y'} = \frac{(1+\alpha)x}{1+\alpha y} = (1 + \alpha)(1 + \alpha) \frac{x}{y}$$

当流动池中的价格与外部市场价格不一致时就会有套利者出现，套利者监控全球各家交易所的价格，一旦发现两家交易所存在价差，就会同时在两边低买高卖赚取中间差价。由于套利者的存在，才使得 Uniswap 币价不会与全球市场脱节。

- Uniswap 中的价格高于外部市场时，套利者从外部市场低价买入，然后高价卖给 Uniswap
- Uniswap 中的价格低于外部市场时，套利者从 Uniswap 低价买入，然后高价卖给外部市场



直观案例：

我在 Uniswap 中新建一个 ETH 与 X 代币的流动池（免费创建，无需费用），创建时 ETH 和 X 的数量分别为 10 和 100，这个时候 X 的价格为 0.1ETH，两者数量的乘积为 1000。创建完成之后，用户就可以在 Uniswap 上进行 ETH 与 X 的兑换了。假设有人用 1 个 ETH 来买 X，这个时候 1ETH 进入流动池，ETH 的数量变为 11，那么要保持两者乘积不变，X 的数量就要减少，这个减少的 X 的数量即为 1 个 ETH 能买到的 X 数量。根据恒定乘积可得， $10 \times 100 = (10 + 1) \times (100 - X')$ ，计算的 $X' = 9.09$ ，即 1 个 ETH 可以买到 9.09 个 X，相对于原来 $1\text{ETH} = 10X$ 的价格来说，滑点（价格误差）为 $(10 - 9.09) / 10 \times 100\% = 9.09\%$ 。将 1 个 ETH 改成 5 个，同样可以算出 5 个 ETH 可以买到 33.33 个 X，平均 1 个 ETH 只能够买到 6.67 个 X，滑点为 33.33%。



总结：

- 优点：

- 完全去中心化，不依赖第三方，任何人都可以基于 uniswap 构建自己的应用
- 可以自由地创建任何 ERC20 交易对
- 与其他去中心化交易所相比 gas 费用更低
- 任何人都可以向 uniswap 流动池中提供流动性，人人都可以是做市商

- 缺点：

- 需要依赖套利者去平衡 uniswap 与外部交易所之间的汇率
- 对大户不友好，进行大额的交易会造成很大的滑点
- 与中心化交易所相比手续费偏高

第二次作业：Build Your First DApp

去中心化学生社团组织治理网站 [参考内容：<https://github.com/LBruyne/ZJU-blockchain-course-2022>]

参与方：一个社团组织中的学生们

需求：为我校某社团实现一个去中心化应用网站，在网站中：

1. 每个学生初始可以通过某种方式拥有或领取一些通证积分（ERC20）。
2. 每个学生可以在应用中可以：
 1. 使用一定数量通证积分，发起关于该社团进行活动或制定规则的提案（Proposal）。
 2. 提案发起后一定时间内，使用一定数量通证积分可以对提案进行投票（赞成或反对，限制投票次数），投票行为被记录到区块链上。
 3. 提案投票时间截止后，赞成数大于反对数的提案通过，提案发起者作为贡献者可以领取一定的积分奖励。
3. （Bonus）发起提案并通过3次的学生，可以领取社团颁发的纪念品（ERC721）

作业内容和提交要求：

- 为了降低框架搭建难度，让大家专注于代码实现，请在Github上fork助教提供的代码框架：<https://github.com/LBruyne/ZJU-blockchain-course-2022>，阅读和按需修改（当然，如果想自己重新搭建自己喜欢的框架，也是可以的）；
- 需要完成的内容：
 - 完成上述需求的合约、前端代码实现；
 - 在Github仓库文件夹内的README.md（markdown文件）中撰写清晰完整的文档，内容包括：
 1. 简单但清晰的功能和实现分析
 2. 如何运行你的项目
 3. 项目运行成功的关键界面和流程截图，主要包括操作流程和与区块链交互的截图
 - 录制一段你的项目从运行到完成整个操作流程的清晰视频
- 提交方式：将自己代码所在Github仓库链接和项目视频发到邮箱：22221263@zju.edu.cn。视频文件名和邮箱主题均为学号 姓名 作业2
- 严禁抄袭，如有发现，会有很严重的Penalty
- 作业截止时间DDL：11.7晚上



浙江大学
ZHEJIANG UNIVERSITY

谢谢